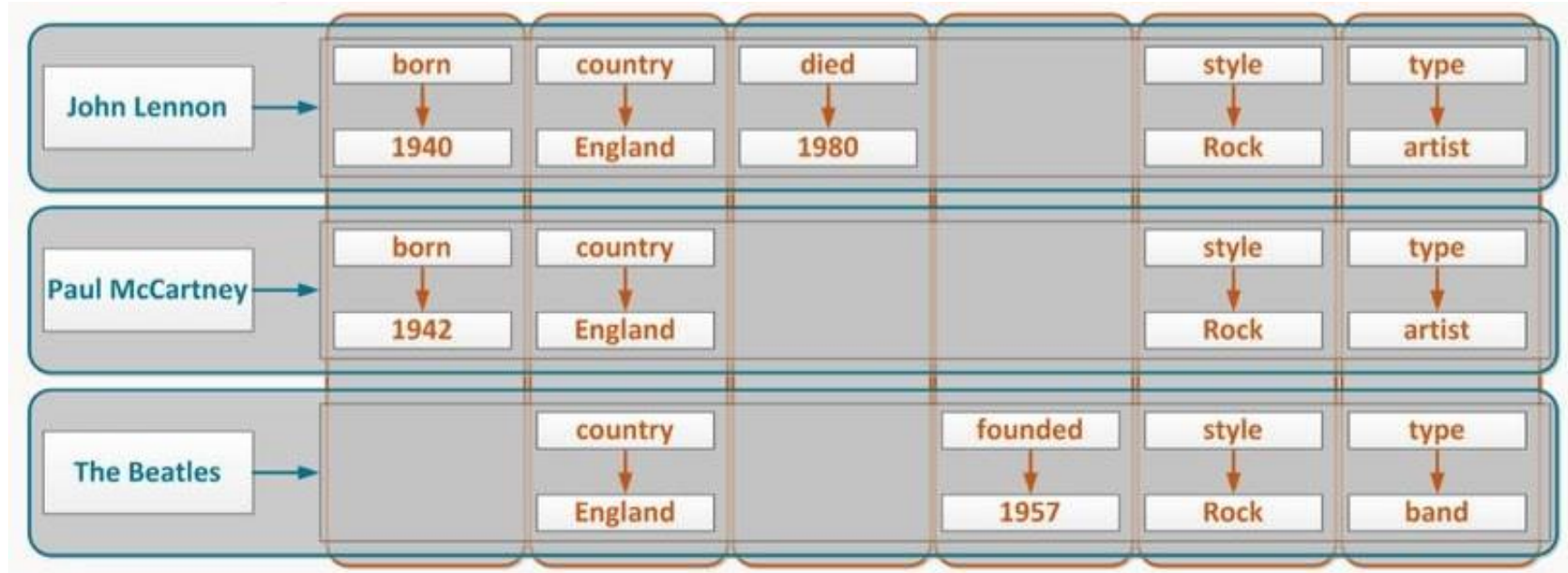




Bases de Datos Tabulares

BASES DE DATOS TABULARES



También llamadas bases de datos *columnares*, bases de datos de *columnas extendidas* o bases de datos *orientadas a columnas*. Almacenan los datos en tablas que pueden tener una cantidad muy grande de columnas

El modelo de datos está conformado por un *modelo tabular*, donde cada fila puede tener una configuración diferente de *columnas*

Precursor de este tipo de bases de datos: Google BigTable



Bases de Datos Tabulares



HYPERTABLE INC

Las bases de datos tabulares son *buenas* en:

Gestión de tamaño

Cargas de escrituras masivas orientadas al stream

Alta disponibilidad

MapReduce



RDBMS	Tabular
Instancia	Cluster
Base de Dato	KeySpace
Tabla	Familia de Columnas
Fila	Fila
Columna	Columna (distinto por cada fila)

Terminología RDBMS vs Tabular

Bases de Datos Tabulares

- ▶ Estructura Libre (puede variar entre registros de la misma familia de columnas)
- ▶ Buena para registrar eventos (logs de aplicaciones).
- ▶ Las familias de columnas pueden almacenar datos, etiquetas, categorías, vínculos, etc.
- ▶ Pueden hacer referencia a otros keySpace.
- ▶ Permiten contar y clasificar accesos y posteriormente calcular análisis estadísticos.

Cuando NO es conveniente utilizarlas

- ▶ Sistemas que requieran transacciones ACID para lecturas y escrituras.
- ▶ Consultas de agregación de datos (sum o avg) ya que deben hacerse del lado del cliente.
- ▶ No es ideal para prototipos tempranos donde no se tenga seguridad sobre los patrones de las consultas
- ▶ Cambios costosos del esquema para consultas existentes.



cassandra

□ include secondary database models

13 systems in ranking, May 2021

Rank			DBMS	Database Model	Score		
May 2021	Apr 2021	May 2020			May 2021	Apr 2021	May 2020
1.	1.	1.	Cassandra +	Wide column	110.93	-3.92	-8.22
2.	2.	2.	HBase +	Wide column	43.24	-0.92	-6.48
3.	3.	3.	Microsoft Azure Cosmos DB +	Multi-model i	34.71	+1.19	+4.03
4.	4.	4.	Datastax Enterprise +	Wide column, Multi-model i	7.55	-0.44	-0.07
5.	5.	5.	Microsoft Azure Table Storage	Wide column	4.29	-0.41	-0.39
6.	6.	6.	Accumulo	Wide column	3.88	-0.23	+0.21
7.	7.	7.	Google Cloud Bigtable	Wide column	3.11	-0.28	+0.93
8.	8.	8.	ScyllaDB +	Multi-model i	2.14	+0.08	+0.24
9.	9.	9.	HPE Ezmeral Data Fabric	Multi-model i	0.86	+0.01	+0.26
10.	10.	↑ 11.	Elassandra	Wide column, Multi-model i	0.56	-0.03	+0.32
11.	11.		Amazon Keyspaces	Wide column	0.45	+0.01	
12.	12.	↓ 10.	Alibaba Cloud Table Store	Wide column	0.37	-0.03	+0.04
13.	13.		SWC-DB	Wide column, Multi-model i	0.00	±0.00	

Cassandra



- ▶ Dentro de los nuevos sistemas de almacenamiento que están surgiendo dentro del universo Big Data, Cassandra es uno de los más interesantes
- ▶ Cassandra se define como una base de datos NoSQL distribuida y masivamente escalable
- ▶ Además, Cassandra introduce conceptos muy interesantes como el soporte para multi data center o la comunicación peer-to-peer entre sus nodos

Historia y orígenes

- ▶ El desarrollo inicial de Cassandra tiene su origen en Facebook, que lo diseñó para potenciar la funcionalidad de búsqueda en el inbox
- ▶ En 2008 fue liberado como proyecto open source y en febrero de 2010 se convirtió en un proyecto top-level de la fundación Apache
- ▶ Está inspirado e influenciado por los papers de Amazon Dynamo de 2007 y de Google BigTable de 2006
- ▶ Hoy en día está mantenido y desarrollado por la compañía Datastax
- ▶ Su nombre está inspirado por la sacerdotisa Cassandra de la mitología griega, que tenía el don de la profecía, y predijo el engaño del Caballo de Troya

Arquitectura y características

- ▶ Cassandra nos proporciona tolerancia a particiones y disponibilidad, pero a cambio de ser eventualmente consistente, tal y como define el teorema CAP
- ▶ El nivel de consistencia puede ser configurado, según nos interese, incluso a nivel de query
- ▶ Es **distribuida**, lo quiere decir que la información está repartida a lo largo de los nodos del cluster. Además ofrece alta disponibilidad, de manera que si alguno de los nodos se cae el servicio no se degradará
- ▶ **Escala linealmente**, lo que quiere decir que el rendimiento crece de forma lineal respecto al número de nodos que añadamos. Por ejemplo, si con 2 nodos soportamos 100.000 operaciones por segundo, con 4 nodos soportaremos 200.000. Esto da mucha predictibilidad a nuestros sistemas
- ▶ **Escala de forma horizontal**, lo que quiere decir que podemos escalar nuestro sistema añadiendo nuevos nodos basados en hardware de bajo costo

Arquitectura y características

- ▶ Implementa una arquitectura Peer-to-Peer, lo que elimina los puntos de fallo único y no sigue patrones maestro-esclavo como otros sistemas de almacenamiento
- ▶ De esta manera cualquiera de los nodos puede tomar el rol de coordinador de una query. Será el driver el que decida qué nodo quiere que sea el coordinador
- ▶ Los datos son repartidos a lo largo del cluster en base a un token único calculado para cada fila por una función hash.
- ▶ Los nodos se reparten equitativamente el rango de tokens que va de -2^{63} a 2^{63} , esto define el nodo primario. Internamente Cassandra replicará los datos entre los nodos con la política que le definamos, por ejemplo definiendo el factor de replicación
- ▶ Además soporta el concepto de data center para agrupar los nodos lógicamente y tener los datos más cerca del usuario

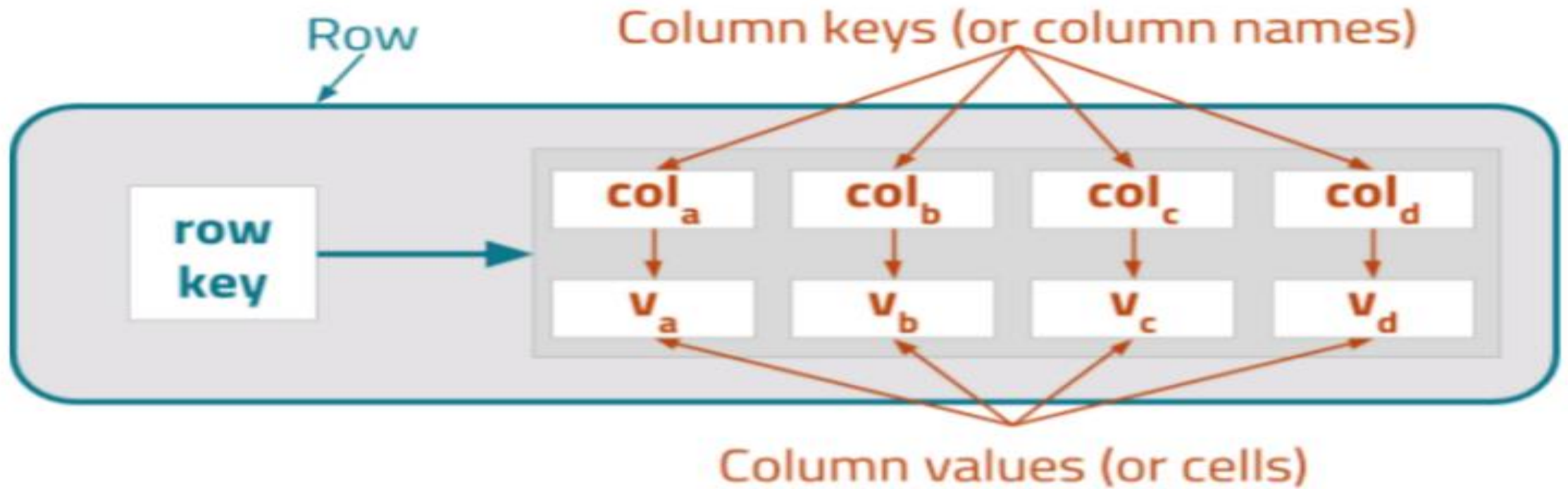
Lenguaje CQL

- ▶ Cassandra Query Language (CQL) es el lenguaje de acceso a datos en Cassandra, es un derivado reducido de SQL
- ▶ En Cassandra los datos están desnormalizados de manera que el concepto de joins o subqueries no existe
- ▶ Podemos interactuar con Cassandra mediante CQL a través de la shell. de CQL, cqlshell
- ▶ También podemos usar herramientas gráficas como DevCenter o a través de los drivers soportados para múltiples lenguajes de programación

Modelado de datos

- ▶ También combina propiedades de una base de datos clave-valor y una orientada a columnas
- ▶ Como podemos ver en el siguiente diagrama la información se organiza de manera que toda fila tiene una clave única y una serie de pares de clave, valor de columna
- ▶ Es importante tener en mente estas características a la hora de diseñar nuestro modelo de datos

Modelado de datos

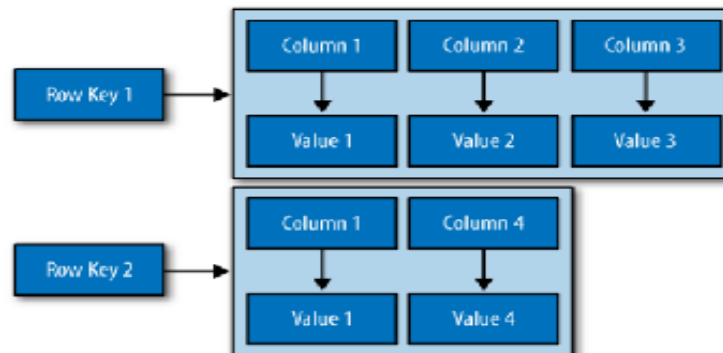


Modelado de datos

- ▶ Cuando diseñemos nuestro modelo debemos guiarnos por el **patrón de acceso a los datos**, hay que hacer un análisis de las queries que pretendemos ejecutar contra nuestro sistema y de esta forma podremos diseñar un modelo eficiente que pueda sacar partido de las ventajas de Cassandra
- ▶ Es importante también **definir adecuadamente la clave de partición** de nuestro datos, ya que Cassandra se basará en esta clave para distribuir los datos a lo largo del cluster. Si queremos aprovechar nuestro cluster debemos pensar en distribuir los datos para evitar cuellos de botella
- ▶ También es recomendable, dado nuestro patrón de consulta, intentar **minimizar el número de particiones** a las que hay que acceder durante una lectura

Cuándo Cassandra es la mejor solución

- ▶ La alta disponibilidad es una de las características principio
- ▶ Su arquitectura distribuida y la manera en que se almacenan los datos va a permitir tener una alta disponibilidad de la base de datos, evitando caídas que pueden suponer pérdida de dinero o pérdida de datos
- ▶ Si tenemos grandes cantidades de datos y preferimos la velocidad para acceder a los datos a la normalización de estos, entonces **Cassandra** también puede ser la base de datos a elegir. Pero cuidado, si estás acostumbrado a trabajar con bases de datos relacionales deberás cambiar la forma de pensar a la hora de diseñar tu modelo de datos



Cuándo Cassandra es la mejor solución (cont.)

- ▶ En el mundo relacional estamos acostumbrados a pensar en relaciones de tablas y la normalización de estas. Sin embargo, en el mundo **Cassandra**, hay que pensar en el almacenamiento en columnas. De modo que lo que llamaríamos una tabla, se va a convertir en una familia de columnas
- ▶ Para acceder de forma más rápida y eficaz manejando grandes volúmenes de datos, diseñaremos familias de columnas sin importar el número de éstas. Así pues, podemos tener un registro en una familia de columnas que cuente con 50 columnas, y otro registro de la misma familia que tenga 300 columnas, ya que accederemos por el llamado Row Key

Cuándo Cassandra es la mejor solución (cont.)

- ▶ Es cierto que quizás la parte más compleja de la adopción de esta base de datos sea cambiar la forma de pensar a la hora de diseñar el modelo, pues éste se debe basar en las consultas que haremos en la aplicación, y no en la relación de datos entre sí
- ▶ Recuerda, en Cassandra no existen los JOIN entre tablas

Ejemplo de CQL

```
CREATE KEYSPACE MiEspacioClaves
  WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' : 3 };

USE MiEspacioClaves;

CREATE COLUMNFAMILY MisColumnas (id text, Apellido text, Nombre text, PRIMARY KEY(id));

INSERT INTO MisColumnas (id, Apellido, Nombre) VALUES ('1', 'Perez', 'Juan');

SELECT * FROM MisColumnas;
```

Resultado:

id	nombre	apellido
1	Juan	Perez

(1 fila)



Usa Cassandra como base de datos de back-end para su servicio de streaming



Funciones orientadas al usuario, pila de análisis y monitoreo interno
Para el 2017: 3000 nodos



Sistema de búsquedas en la bandeja de entrada



Implementó Cassandra para facilitar el escalamiento horizontal



En 2012 Walmart invirtió en la creación de 10 clústers formados por 250 nodos de procesamiento

Principales Clientes

Cassandra

- Iniciado por Facebook
- Código abierto en 2008
- Proyecto apache en 2009
- Licencia: Apache License 2.0
- Escrito en Java
- Multiplataforma
- Soporta terabytes de datos y millones de ficheros.

Cassandra

- **Esquema dinámico:** El esquema que define la estructura de los datos puede cambiar en tiempo de ejecución.
- **No hay un único punto de fallo:** Los datos se replican automáticamente a varios nodos. Perder un nodo no causa la baja del clúster.
- **Alta disponibilidad:** Los datos están disponibles la mayor parte del tiempo gracias a la redundancia que introduce la replicación de datos.

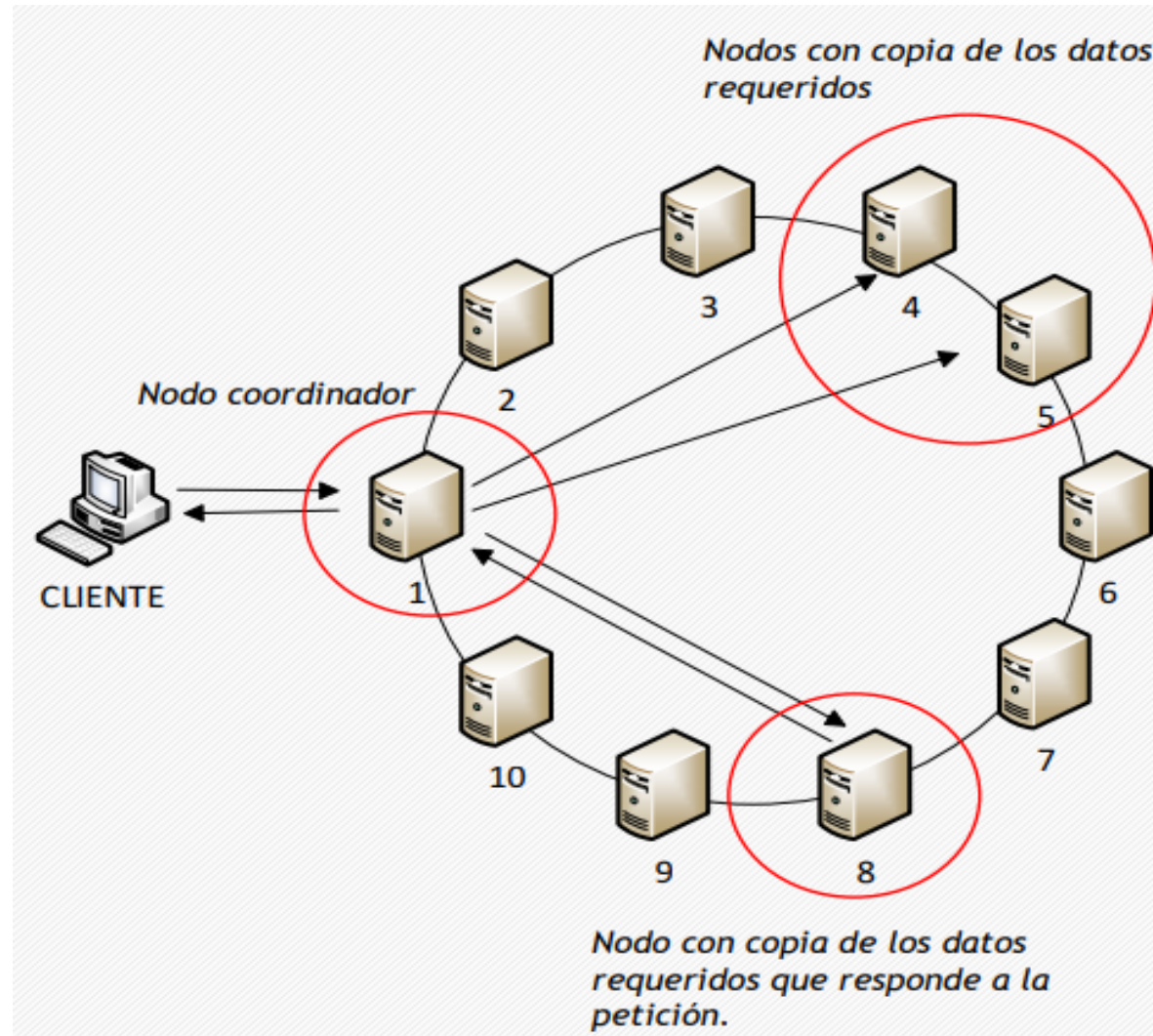
Cassandra

- ***Particionado de los datos:*** La topología de ***Cassandra*** es la de un anillo a través del cual se distribuyen los datos para minimizar cuellos de botella en el acceso a los mismos.
- ***Escalabilidad horizontal:*** Hasta un alto número de máquinas la capacidad de cómputo aumenta linealmente con el número de máquinas.
- ***Capacidad para manejar cientos de gigabytes de datos.***

Arquitectura de Cassandra

Varios nodos independientes comunicados mediante un protocolo P2P (peer-to-peer):

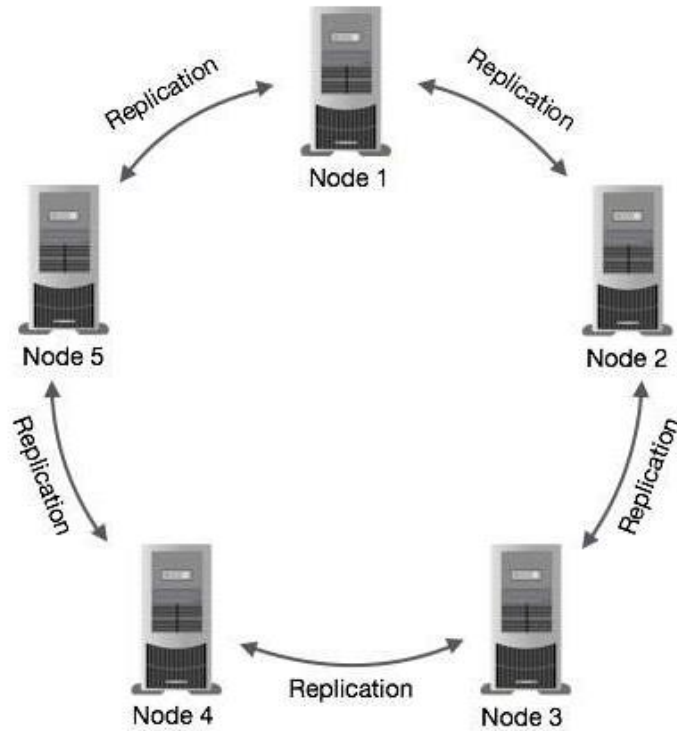
- *Todos los nodos intercambian*
- *información con el resto de forma continua.*



Arquitectura de Cassandra

- ▶ No hay nodos “principales”. Los clientes pueden conectarse a cualquiera de los nodos para realizar las operaciones de lectura y escritura:
- ▶ *El nodo al que se conecta el cliente actúa como coordinador entre éste y el resto de los nodos en dónde se encuentran los datos afectados por la consulta.*
- ▶ *El coordinador determina qué nodos deben responder a la consulta.*

Replicación en Cassandra



Uno o más nodos de un clúster actúan como réplicas de un dato dado.

Si se detecta que algunos de los nodos respondieron con un valor desactualizado, devolverá el valor más reciente al cliente.

Replicación en Cassandra

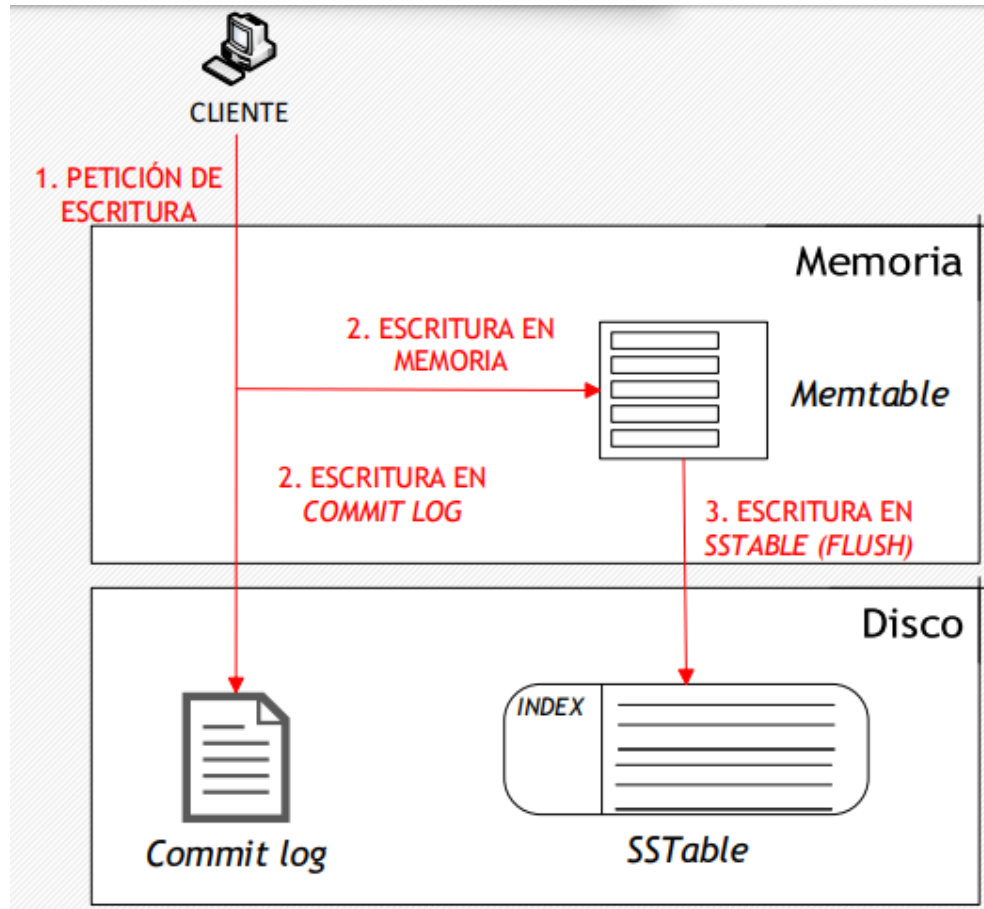
- ▶ Después de devolver el valor más reciente, Cassandra realiza una reparación de lectura en segundo plano para actualizar los valores obsoletos.
- ▶ Utiliza el Protocolo **Gossip** (*chisme*) en segundo plano para permitir que los nodos se comuniquen entre ellos y detecten cualquier nodo defectuoso en el clúster.
- ▶ **Gossip** : Procedimiento o proceso de comunicación entre pares basado en la forma en que se propagan las epidemias (el vecino le informa los cambios)

Elementos de Cassandra

- ▶ **Nodo:** Es una instancia de Cassandra que almacena datos y responde a las solicitudes de lectura/escritura. Cada nodo es responsable de una parte específica del conjunto de datos. En términos simples, un nodo es un servidor que forma parte del sistema distribuido y tiene una copia de los datos que le corresponde según el algoritmo de partición de la base de datos.
- ▶ **Data Center:** Es un conjunto de nodos agrupados lógicamente. Un data center puede estar ubicado en una única ubicación física o distribuido geográficamente para replicación y alta disponibilidad. En Cassandra, los data centers se utilizan principalmente para separar nodos por motivos de rendimiento o para replicación en distintas zonas geográficas. Puedes tener diferentes data centers para manejar cargas de trabajo distintas, como uno para operaciones de análisis y otro para procesamiento en tiempo real.
- ▶ **Clúster:** Es el conjunto completo de todos los nodos de Cassandra organizados bajo una configuración común. Un cluster incluye uno o más data centers y está diseñado para brindar redundancia, alta disponibilidad y escalabilidad. Los nodos en un cluster trabajan juntos para almacenar y replicar datos, lo que asegura la resistencia a fallas.

Elementos de Cassandra

- ▶ ***Commit log*** : el registro de confirmación (*Commit log*) es un mecanismo de recuperación de fallas en Cassandra. Cada operación de escritura se escribe en el registro de confirmación.
- ▶ ***MemTable***: es una estructura de datos residente en la memoria. Después del registro de confirmación (*Commit log*), los datos se escribirán en la *SStable* desde *MemTable*.
- ▶ ***SStable***: es un archivo de disco en donde se guarda el contenido de la *MemTable* cuando alcanza un valor determinado. Cada fichero *SStable* es inmutable una vez creado.
- ▶ ***Filtros de Bloom***: no son más que algoritmos rápidos, no deterministas, para probar si un elemento es miembro de un conjunto. Es un tipo especial de caché. Se accede a los filtros Bloom después de cada consulta.



Actividad de Escritura

Actividad de Escritura

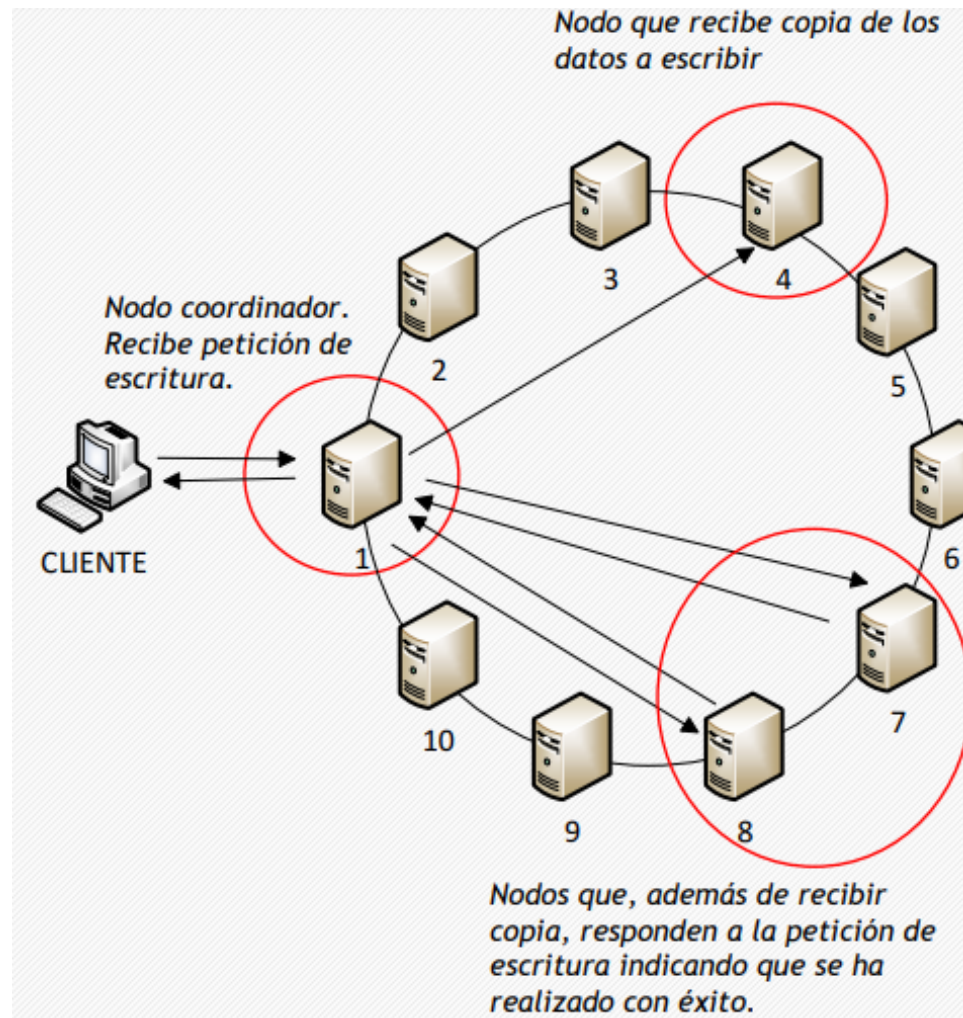
- ▶ Escritura en ***Commit log***: almacena ***TODA*** la actividad de escritura para garantizar la durabilidad de los datos.
- ▶ Escritura en ***MemTable***: almacena las escrituras de cada familia de columnas de forma temporal en memoria.
 - ▶ *En Cassandra, las escrituras son “baratas”.*
 - ▶ *Si el sistema se apaga, se recupera desde el **Commit log***
- ▶ Flush de la ***MemTable***: cuando se llena se libera la memoria mediante un borrado de los datos.

Actividad de Escritura

- ▶ Escritura en *SSTable*: los datos de la *MemTable* se escriben en ficheros *SSTable* antes de borrar la *MemTable*.
- ▶ Los ficheros *SSTable* son inmutables, no se vuelve a escribir en ellos después de volcar los datos de la *MemTable*, por ello, los datos de una misma partición pueden estar repartidos en varias *SSTable*.
- ▶ Este es el motivo por el que las lecturas son más “caras” que las escrituras en Cassandra, ya que una lectura suele requerir el acceso y búsqueda en varias *SSTable*.

Actividad de Escritura

- ▶ Compactación: Ejecutada periódicamente por Cassandra, se trata de reducir el número de ficheros *SSTable*, eliminando los datos antiguos.
 - ▶ *El objetivo es hacer más eficientes las lecturas.*



Actividad de Escritura (Cluster)

Actividad de Escritura (Cluster)

- ▶ El nodo coordinador envía una petición de escritura a todas las réplicas que contengan la fila a ser escrita.
- ▶ Los datos se escriben en todos los nodos disponibles que reciben la petición.
- ▶ El nivel definido de consistencia determina cuántos nodos han de responder a la petición para determinar que la escritura se ha realizado con éxito.

Actividad de Escritura (Cluster)

- ▶ El éxito de la escritura significa que el dato ha sido escrito en el Commit log y el MemTable.
- ▶ Todos los nodos pueden ser coordinadores en una petición.
- ▶ La selección del nodo coordinador en cada petición se realiza en base a una política predefinida en la Base de Datos.

Peticiones de Borrado

- ▶ Cuando una fila es eliminada, no se borra del disco (*SSTables* inmutables), sino que se marca como eliminada (tombstone).
- ▶ Durante la compactación, las filas marcadas son eliminadas definitivamente del disco.
- ▶ Tiempo de gracia: si un nodo que contiene la fila a ser eliminada se encuentra caído en ese momento, aún tiene la posibilidad de recibir la orden de eliminación si se recupera en un intervalo de tiempo predefinido.

Peticiones de Borrado

- ▶ Si no se recupera en ese tiempo, la fila no será marcada como eliminada, por lo que pueden existir inconsistencias en las réplicas. Por ello, se recomienda que los administradores realicen cada cierto tiempo tareas de mantenimiento en Cassandra.
- ▶ LLTS: los datos pueden tener una “fecha de caducidad”, a partir de la cual son marcados por Cassandra como eliminados.

Peticiones de Lectura

- ▶ En cada nodo que recibe la lectura, se accede mediante *key* a la *MemTable* para recuperar los datos requeridos.
- ▶ Si los datos (o parte de ellos) no se encuentran en la *MemTable*, se accede a las *SSTable*.
- ▶ Si no se ha realizado la tarea de compactación recientemente, es probable que haya que leer en varias *SSTables* para recuperar todas las filas requeridas (Problema de lentitud)
- ▶ Existen *tres* mecanismos distintos para la obtención del resultado

Peticiones de Lectura

- ▶ Se define un nivel de consistencia (*Consistency Level*) en la recuperación de información, es decir se define cuantos nodos son necesarios que respondan para asumir que la respuesta es consistente.
- ▶ Existen tres mecanismos distintos para la obtención del resultado:
 - ▶ *Direct read request*: el nodo coordinador contacta con un nodo que contenga la réplica requerida y la recupera.

Peticiones de Lectura

- ▶ ***Background read repair request:*** en caso de existir inconsistencias en las réplicas, las réplicas con un timestamp más antiguo son sobrescritas con los datos de la réplica “más actual”.
- ▶ ***Digest request:*** contacta con tantos nodos con réplicas como se determine en el nivel de consistencia. Se comprueba la consistencia de los datos retornados por el nodo contactado mediante ***Direct read request***.

Peticiones de Lectura

► *Digest request (continuación)*

- Se envía la consulta aquellos nodos que estén respondiendo “más rápido”.
- Si existen inconsistencias, se consideran como válidos las réplicas con un timestamp más reciente. Estos serán los retornados al cliente.
- Una vez contactados los nodos determinados por el nivel de consistencia, se envía un Digest request al resto de réplicas, para determinar si son también consistentes.

Consistencia

Consistencia reconfigurable (***consistency_level***)

- Al escribir, determina en cuántas réplicas se debe escribir para confirmar a la aplicación cliente.
- Al leer, se especifica cuántas réplicas deben responder para retornar los datos a la aplicación cliente.
- Los datos se guardan en un Clúster o Ring donde se asignan datos a los nodos dentro de un anillo.
- Un nodo tiene réplicas para diferentes rangos de datos. Si un nodo se cae, su réplica puede responder.
- Un protocolo P2P hace que los datos se repliquen entre nodos acorde a un factor de replicación definido.

Consistencia

- ▶ Se ofrece la elección del nivel de consistencia para las lecturas y escrituras. Un ejemplo posible de consulta utilizando consistencia es el siguiente:

```
SELECT * FROM users WHERE state='TX' USING CONSISTENCY QUORUM;
```

Consistencia de lectura (Read)	
Nivel	Descripción
ONE / TWO / THREE	Devuelve una respuesta de una/dos/tres réplicas más cercanas, según lo determinado por el coordinador. Por defecto, se ejecuta una “reparación de lectura” en segundo plano para hacer que las otras réplicas sean consistentes.
QUORUM	Devuelve el registro después de un quórum cuando $n/2+1$ réplicas hayan respondido.
LOCAL_QUORUM	Devuelve el registro después de un quórum cuando $n/2+1$ réplicas hayan respondido en el mismo <i>datacenter</i> que el nodo coordinador. Evita la latencia de comunicación entre el <i>datacenter</i> .
EACH_QUORUM	Devuelve el registro una vez que el quórum de réplicas en cada <i>datacenter</i> de la agrupación haya respondido.
ALL	Devuelve el registro después de que todas las réplicas hayan respondido.

$$\text{Quorum} = (\text{sum_of_replication_factors} / 2) + 1$$

Consistencia

Consistencia en escritura (Write)	
Nivel	Descripción
ANY	Una escritura debe ser escrita en al menos un nodo disponible.
ONE / TWO / THREE	Una escritura debe escribirse en el registro de <code>commit</code> y <code>memtable</code> de al menos un/dos/tres nodos de réplica.
QUORUM	Una escritura debe escribirse en el registro de <code>commit</code> y <code>memtable</code> en un quórum de nodos de réplica. Proporciona una fuerte consistencia si se puede tolerar cierto nivel de fracaso. Se determina por $(\text{replication_factor}/2)+1$.
LOCAL_QUORUM	Una escritura debe escribirse en el registro de <code>commit</code> y <code>memtable</code> en un quórum de nodos de réplica en el mismo <i>datacenter</i> que el nodo coordinador. Evita la latencia de comunicación entre el <i>datacenter</i> .
EACH_QUORUM	Consistencia fuerte. Una escritura debe escribirse en el registro de <code>commit</code> y <code>memtable</code> en un quórum de nodos de réplica en todos los <i>datacenter</i> .
ALL	Una escritura debe escribirse en el registro de <code>commit</code> y <code>memtable</code> de todos los nodos de réplica en el cluster. Proporciona la más alta consistencia y la menor disponibilidad de cualquier otro nivel.

Consistencia

- ▶ Ante la posibilidad de encontrar datos inconsistentes en los nodos a pesar de **tombstones**, se recomienda realizar tareas rutinarias de mantenimiento.
- ▶ Existe una operación llamada repair, que puede utilizarse por parte del usuario para asegurar que todos los nodos son consistentes



Seguridad

- ▶ Seguridad a nivel de usuarios: logins con passwords y permisos de gestión y administración vía GRANT/REVOKE.
- ▶ Opciones de encriptación tanto entre clientes y clústeres como entre nodos.
- ▶ Ofrece varias opciones de backup. Recomendado hacerlos de forma regular ante errores como los borrados accidentales.
- ▶ Existen herramientas externas (DataStax Enterprise) que ofrecen opciones avanzadas de seguridad (autenticación externa, encriptación de tablas, y auditoría de datos).

Seguridad (cont.)

► Roles

- CREATE ROLE alice WITH PASSWORD = 'password_a' AND LOGIN = true;
- GRANT report_writer TO alice;
- REVOKE report_writer FROM alice;
- LIST ROLES;

► Users

- CREATE USER alice WITH PASSWORD 'password_a' SUPERUSER;

► Permissions

- GRANT SELECT ON ALL KEYSPACES TO data_reader;
- REVOKE SELECT ON ALL KEYSPACES FROM data_reader;
- LIST ALL PERMISSIONS ON keyspace1.table1 OF bob;

Estructura de Datos

- ▶ Columna (***Columns***): unidad básica de almacenamiento. Consistente en una estructura de tres valores: ***name***, ***value*** y ***timestamp***.
- ▶ Super Columna(***SuperColumns***): se consideran *Columns* que almacenan *subcolumns*. Son opcionales (se desaconseja su uso).
- ▶ Filas o entradas (***Rows***): conjunto de columnas de una familia con valor asignado.

Estructura de Datos

- ▶ Familia de Columnas (*ColumnFamilies*): contenedor de múltiples columnas. Es equivalente a una tabla en el modelo relacional. Cada *fila* se identifica y accede a ella mediante una *ClaveDeFila*
- ▶ *Keyspace*: es la agrupación de *Familia de Columnas*, normalmente hay un *keyspace* por aplicación.
- ▶ *Clúster*: las máquinas (nodos) de una instancia de Cassandra. Los *clústers* pueden contener múltiples *keyspaces*.

Columna

Estructura:

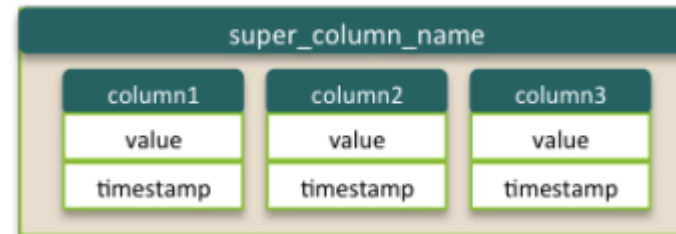
- **Name**: nombre de la columna.
- **Value**: valor almacenado en la columna.
- **Timestamp**: fecha y hora de escritura.

```
struct Column {  
    1: binary name,  
    2: binary value,  
    3: i64 timestamp  
}
```

Supercolumnas

- Super Column: es una *column* cuyos *values* son una o más *columns*, que en este contexto se llamarán *subcolumns*. Las *subcolumns* están ordenadas y el número de columnas que se puede definir es ilimitada. Las *supercolumns*, a diferencia de las *columns*, no tienen un timestamp definido. Además no son recursivas, es decir, solamente tienen un nivel de profundidad. Esquema visual del modelo de datos:

```
Supercolumna (  
  Nombre de la columna -> "XXX" (  
    columna1 -> "XXX" (  
      Nombre -> "Nombre del campo"  
      Valor -> "Valor del campo"  
      Timestap -> "Marca de tiempo"  
    )  
    columna2 -> "XXX" (  
      Nombre -> "Nombre del campo"  
      Valor -> "Valor del campo"  
      Timestap -> "Marca de tiempo"  
    )  
  )  
)
```



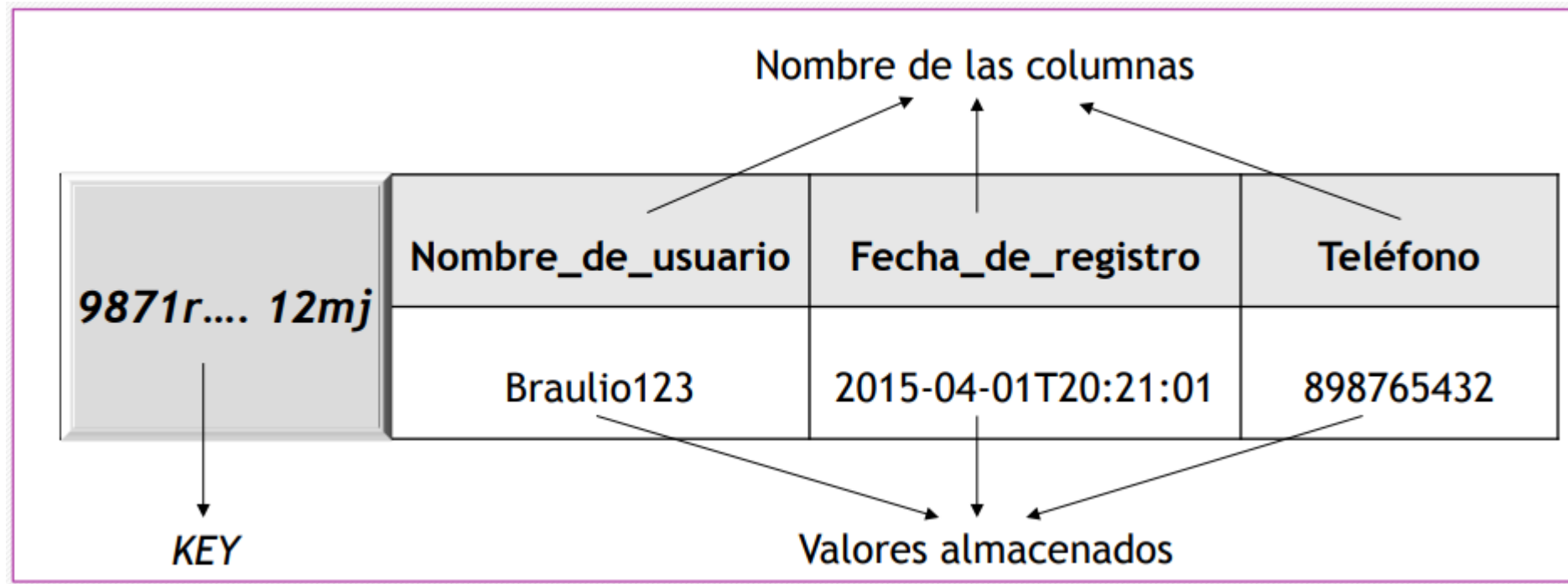
Ejemplo

- Los valores son proporcionados por la aplicación, incluido el timestamp. Por ello, es necesario tener sincronizado el reloj del sistema con el reloj del clúster.
- El timestamp es usado para resolver conflictos (Consistencia Eventual)
- El timestamp puede ser de cualquier tipo, pero la convención marca que sea el valor en microsegundos desde 01/01/1970.

```
{  
  "name": "Nombre_de_usuario",  
  "value": "Braulio123",  
  "timestamp": 987654321  
}
```

Fila

- Agregación de columnas con un nombre para referenciarlo (rowKey).
- Equivalente a la fila en el modelo relacional.



RowKey

La RowKey (RK) es el equivalente a la clave primaria del modelo relacional. Tienen dos partes (claves):

- De particionado (partition key): las filas con el mismo valor de clave de particionado se almacenan en la misma partición del disco (físicamente juntas).
- De agrupamiento (clustering key): determina el orden físico en el que se almacenan las filas.

Familia de Columnas

Es una agregación de filas donde cada *ColumnFamily* es guardada en un fichero separado y ordenado por la RowKey (equivale a las tablas en el modelo relacional).

Tipos de columnas:

- Standard Columns
- Counter Columns
- Collection Columns: SET, LIST, MAP
- User-defined type
- Tuple-type
- Timestamp type

Familia de Columnas

9871r.... 12mj	Nombre_de_usuario	Fecha_de_registro	Teléfono
	Braulio123	2015-04-01T20:21:01	898765432
911u.... 1pop	Nombre_de_usuario	Fecha_de_registro	Teléfono
	Joaquín89	2014-04-02T10:01:01	123764436

KEY

Valores almacenados

KeySpace

- Es equiparable al esquema del modelo relacional, uno por app. Los atributos básicos de un keyspace son:
- **Replication_factor:** Número de nodos que almacenarán una copia de cada fragmento de datos. En el ejemplo, 3 significa que habrá 3 réplicas.
- **SimpleStrategy:** Utilizada para configuraciones simples o de un solo centro de datos
- **NetworkTopologyStrategy:** (varios centros de datos)

```
CREATE KEYSPACE nombre WITH replication =  
    {'class': 'SimpleStrategy', 'replication_factor' : 3};
```

Particionado

Cassandra ofrece soporte para particionado distribuido de datos:

- ***RandomPartitioner***: usa un algoritmo de hash para distribuir las filas de datos aleatoriamente a lo largo del clúster. La idea principal es garantizar un **equilibrio uniforme de los datos** entre todos los nodos del clúster.
- ***OrderPreservingPartitioner***: distribuye las claves de partición de manera que las claves con valores cercanos en el espacio de claves también se almacenan cerca en el mismo nodo o nodos adyacentes. Es decir, mantiene el orden natural de las claves.

Índices Primarios

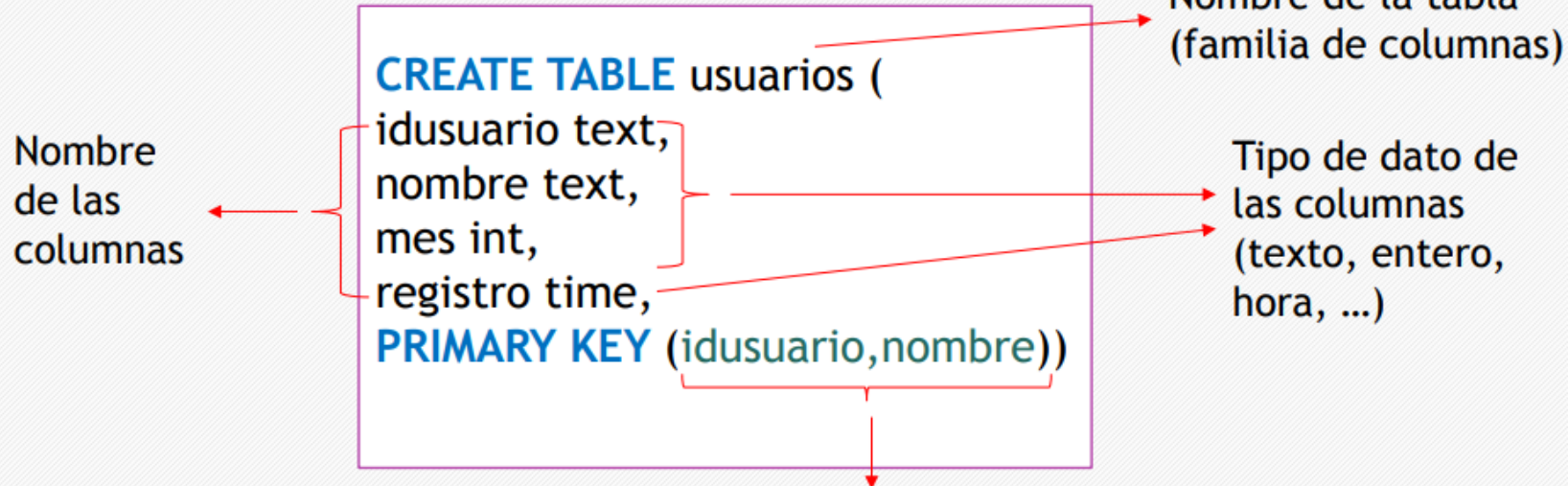
- Los índices primarios son únicos por cada fila.
- Las filas son asignadas a cada nodo por el *partitioner* y el replica *placement strategy* en base a su índice primario.
- Dado que cada nodo conoce qué rango de valores del índice primario almacena, las consultas pueden realizarse de forma eficiente escaneando únicamente los índices de las réplicas de las filas buscadas.

Índices Secundarios

- Son índices sobre columnas.
- Se implementan como una tabla oculta, separada de la tabla que contiene los valores originales.
- No se recomienda su uso para consultas que requieran comprobar un gran volumen de datos para finalmente retornar un pequeño conjunto de resultados

Creación de una tabla

- Ejemplo de creación de tabla (familia de columnas):



Campos que forman parte de la *KEY* de la familia de columnas:

- La primera parte de la *KEY* define la *partition key* o clave de partición. En el ejemplo, “idusuario”.
- La segunda parte de la *KEY* define la *clustering key* o clave de ordenamiento. En el ejemplo, “nombre”.

Clave Primaria

- La *Primary key* contiene las *partition key* y *clustering key* de la tabla:
 - ¡El orden importa!.
 - El primer valor de la *primary key* se toma como *partition key*.
 - El segundo valor, como *clustering key*.

PRIMARY KEY (mes, idusuario)

Partition key *Clustering key*

- Tanto la *partition* como la *clustering key* pueden ser compuestas:

PRIMARY KEY ((idusuario,mes),nombre,registro)

Partition key *Clustering key*

Asignación de Espacio

```
CREATE TABLE usuarios (  
  idusuario text,  
  nombre text,  
  mes int,  
  registro time,  
  PRIMARY KEY (mes,nombre))
```

Partition key *Clustering key*

- Las réplicas de las filas de una partición se encuentran físicamente juntas.
- Las diferentes particiones de las filas de una familia de columnas (tabla) pueden residir físicamente en nodos diferentes

Partition 1 ->

Mes	Nombre	Idusuario	Registro
5	Albert	ap67	21-05-2015
5	John	uj89	02-05-2013

Partition 2 ->

Mes	Nombre	Idusuario	Registro
11	John	jk98	09-11-2013
11	Peter	pk34	08-11-2013

Asignación de Espacio

```
CREATE TABLE usuarios (  
  idusuario text,  
  nombre text,  
  mes int,  
  registro time,  
  PRIMARY KEY (idusuario))  
  Partition key
```

- En este caso, al ser el idusuario único, todas las particiones de la tabla contienen una sola fila.
- No hay *clustering key*.

Partition 1 ->	5	Albert	ap67	21-05-2015
Partition 2 ->	5	John	uj89	02-05-2013
Partition 3 ->	11	John	jk98	09-11-2013
Partition 4 ->	11	Peter	pk34	08-11-2013

Consulta

Proyección. Un * indica que se devuelven todas las columnas.

Tabla (familia de columnas) consultada. Solo se puede consultar una tabla (no hay *JOINS*).

```
SELECT idusuario FROM usuarios WHERE idusuario='1p67' ORDER BY nombre
```

Criterios de búsqueda a cumplir. En el *WHERE*, sólo se pueden utilizar columnas que formen parte de la *partition key* o sobre las que se haya definido un índice secundario. Si la *partition key* es compuesta y se incluye en el *WHERE*, han de incluirse todos sus campos.

Criterios de ordenamiento. Siempre han de ser sobre columnas que estén incluidas en la *clustering key*.

Dada la siguiente definición de tabla:

```
CREATE TABLE usuarios (  
  idusuario text,  
  nombre text,  
  mes int,  
  registro time,  
  PRIMARY KEY ((idusuario, mes), nombre))  
                Partition key Clustering key
```

La siguiente consulta devolvería un error, ya que no se han incluido en el WHERE todas las columnas de la partition_key:

```
SELECT idusuario FROM usuarios WHERE mes=9
```

La siguiente consulta sería correcta, ya que incluye todos los campos de la partition_key en el WHERE:

```
SELECT idusuario FROM usuarios WHERE mes=9  
AND idusuario='agp88'
```

Primary Key in Cassandra

A CQL primary key is composed of two parts:

partition key

- It is the first component of the primary key definition. It can be a single column or, using an additional set of parenthesis, can be multiple columns. A table must have at least one partition key, the smallest possible table definition is:

```
CREATE TABLE t (k text PRIMARY KEY);
```

clustering columns

- The columns are the columns that follow the partition key in the primary key definition. The order of those columns define the *clustering order*.

ie examples of primary key definition are:

- `PRIMARY KEY (a)` : `a` is the single partition key and there are no clustering columns
- `PRIMARY KEY (a, b, c)` : `a` is the single partition key and `b` and `c` are the clustering columns
- `PRIMARY KEY ((a, b), c)` : `a` and `b` compose the *composite* partition key and `c` is the clustering column

Why ALLOW FILTERING?

```
CREATE TABLE blogs (  
  blogId int,  
  time1 int,  
  time2 int,  
  author text,  
  content text,  
  PRIMARY KEY(blogId, time1, time2));
```

If you execute the following query:

```
SELECT * FROM blogs;
```

Cassandra will return you all the data that the table blogs contains. If you now want only the data at a specified time1, you will naturally add an equal condition on the column time1:

```
SELECT * FROM blogs WHERE time1 = 1418306451235;
```

In response, you will receive the following error message:

Bad Request: Cannot execute this query as it might involve data filtering and thus may have unpredictable performance. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING.

Why ALLOW FILTERING?

- ▶ Cassandra knows that it might not be able to execute the query in an efficient way. It is therefore warning you: “Be careful. Executing this query as such might not be a good idea as it can use a lot of your computing resources”.
- ▶ The only way Cassandra can execute this query is by retrieving all the rows from the table blogs and then by filtering out the ones which do not have the requested value for the time1 column.
- ▶ If your table contains for example a 1 million rows and 95% of them have the requested value for the time1 column, the query will still be relatively efficient and you should use ALLOW FILTERING.
- ▶ On the other hand, if your table contains 1 million rows and only 2 rows contain the requested value for the time1 column, your query is extremely inefficient. Cassandra will load 999, 998 rows for nothing. If the query is often used, it is probably better to add an index on the time1 column.
- ▶ Unfortunately, Cassandra has no way to differentiate between the 2 cases above as they are depending on the data distribution of the table. Cassandra is therefore warning you and relying on you to make the good choice.

Materialized Views

```
CREATE MATERIALIZED VIEW monkeySpecies_by_population AS  
SELECT * FROM monkeySpecies  
WHERE population IS NOT NULL AND species IS NOT NULL  
PRIMARY KEY (population, species)  
WITH comment='Allow query by population instead of species';
```

A materialized view cannot be directly updated, but updates to the base table will cause corresponding updates in the view

Functions

- ▶ CQL supports 2 main categories of functions:
 - ▶ **scalar functions** that take a number of values and produce an output

For example the last two days of data can be retrieved using:

```
SELECT * FROM myTable WHERE date >= currentDate() - 2d;
```

- ▶ **aggregate functions** that aggregate multiple rows resulting from a SELECT statement

```
SELECT SUM (players) FROM plays;
```

Triggers

CREATE TRIGGER myTrigger

ON myTable

#(java class)

USING 'org.apache.cassandra.triggers.AuditTrigger'

The actual logic that makes up the trigger can be written in any Java (JVM) language and exists outside the database, i.e. in a **lib/triggers subdirectory** of the Cassandra installation directory

It loads during cluster startup, and exists on every node that participates in a cluster. The trigger defined on a table fires before a requested DML statement occurs, which ensures the atomicity of the transaction

DROP TRIGGER myTrigger;